L S P

Liskov Substitution Principle

Subtypes must be substitutable

DEFINITION

A subtype must be usable anywhere its base type is expected, without breaking behavior.

Substitutability is about behavior, not just shape. A subtype is free to accept more and promise more, but it must never demand more from callers (strengthen a precondition) or deliver less than the base guarantees (weaken a postcondition), and it must preserve the base type's invariants. The compiler checks the shape; LSP is the part it cannot check — the contract behind the methods.

WHY IT MATTERS

Polymorphism is a promise: a caller holding the base type should never need to know which subtype it has. Break that and the symptoms are nasty — distant code starts failing, and people "fix" it with instanceof checks that defeat the whole point of the hierarchy. Honoring LSP is what lets OCP work: extensions are safe only because every subtype is a faithful stand-in for its base.

Vasyl Krupka · Ukraine

How it works

Base contract	The promises of the supertype: preconditions it requires, postconditions it guarantees, invariants it always keeps. Subtypes inherit these as obligations.
Subtype	A drop-in for the base. It may widen what it accepts and narrow what it returns, never the reverse, and must keep every invariant.
Caller	Code written against the base type only. Its correctness must not depend on which subtype it actually receives.

HOW TO APPLY

1. Write the base contract down: what it requires, what it guarantees, what stays invariant.
2. Check each subtype accepts at least as much and returns no less specific (params contravariant, returns covariant).
3. Substitute the subtype everywhere the base is used; if a caller needs a special case, the is-a is false.
4. When behavior really differs, drop inheritance — give both a shared interface and compose instead.

LITMUS TEST

Substitute the subtype everywhere the base type is used. If any caller now needs a special case or breaks, it isn't a true "is-a".

Vasyl Krupka · Ukraine

— Smell → fix

Square passes the Rectangle type check but breaks its behavioral contract:

```
class Rectangle { setW(w){ this.w=w } setH(h){ this.h=h } }
class Square extends Rectangle {
setW(w){ this.w=w; this.h=w } // setter also mutates height
}
// r.setW(5); r.setH(4): caller expects area 20, gets 16
```

↓ MODEL THE DIFFERENCE, DON’T INHERIT IT

```
interface Shape { area(): number }
class Rectangle implements Shape { area(){ return this.w*this.h } }
class Square implements Shape { area(){ return this.s*this.s } }
// no inheritance claim; neither weakens the other's contract
```

Square is not a behavioral Rectangle, so don't make it one. Give both a shared Shape contract and model their differences explicitly.

USE WHEN

- ✓ Designing inheritance hierarchies
- ✓ Validating an "is-a" claim before subclassing

AVOID WHEN

- ▲ Forcing inheritance where behavior really differs — compose instead

— Trade-offs & pitfalls

PROS

+ Polymorphism stays safe
+ Fewer instanceof checks

CONS

− Hard to verify
− Violations are subtle (Square/Rectangle)

COMMON PITFALLS

- ▲ Throwing in an overridden method ("not supported") — that strengthens a precondition and breaks substitutability.
- ▲ Reusing a base class for its fields/shape when the behavioral contract actually differs (Square extends Rectangle).
- ▲ Returning null or a weaker result where the base promised a value — a silently weakened postcondition.

WHERE YOU SEE IT

- ▸ A read-only list that extends a mutable one and throws on add — callers expecting to add break at runtime.
- ▸ A Stream or Iterator subtype that cannot reset where the base contract allows it.
- ▸ An ORM entity subclass that tightens validation, so saving it through a base reference suddenly fails.

SEE ALSO

OCP	substitutable subtypes are what make safe extension possible
CoI	when is-a fails the contract, prefer has-a composition
ISP	narrow contracts are easier to honor fully
DIP	callers depend on the abstraction LSP keeps trustworthy

Vasyl Krupka · Ukraine

Interview & sources

State LSP precisely.

A subtype must not strengthen preconditions or weaken postconditions, and must preserve the base type’s invariants and history constraint.

Why is Square/Rectangle the classic case?

Both are valid data shapes, but Square’s coupled setter weakens a postcondition callers rely on — a behavioral violation the compiler cannot catch.

How do variance rules relate?

To stay substitutable, method parameters should be contravariant (accept at least as much) and return types covariant (return no less specific).

How do you detect LSP violations in practice?

Run the base type’s test suite against every subtype (contract tests). If a subtype fails tests written for the base, it is not substitutable.

REMEMBER

If S is a subtype of T, code written for T must keep working when handed an S.

SOURCES

Barbara Liskov, Jeannette Wing — "A Behavioral Notion of Subtyping" (ACM TOPLAS, 1994)

Barbara Liskov — "Data Abstraction and Hierarchy" (OOPSLA keynote, 1987)